

### 4.2.1

1

```
((lambda (x) (x (y x))) z)
= (x (y x)) [z <- x]
= (x [z <- x] (y x) [z <- x])
= (z (y [z <- x] x [z <- x]))
= (z (y z))
```

2

```
((lambda (x) (x y)) (lambda (y) (x y)))
= (x y) [(lambda (y) (x y)) <- x]
= (x [(lambda (y) (x y)) <- x] y [(lambda (y) (x y)) <- x])
= ((lambda (y) (x y)) y)
= (x y) [y <- y]
= (x y)
```

3

```
((lambda (x) (lambda (y) ((x y) z))) (lambda (a) y))
= (lambda (y) ((x y) z)) [(lambda (a) y) <- x]
= (lambda (w) (((x y) z) [w <- y]) [(lambda (a) y) <- x])
= (lambda (w) ((x y) [w <- y] z [w <- y]) [(lambda (a) y) <- x])
= (lambda (w) ((x [w <- y] y [w <- y]) z) [(lambda (a) y) <- x])
= (lambda (w) ((x w) z) [(lambda (a) y) <- x])
= (lambda (w) ((x w) [(lambda (a) y) <- x] z [(lambda (a) y) <- x]))
= (lambda (w) ((x [(lambda (a) y) <- x] w [(lambda (a) y) <- x]) z [(lambda (a) y) <- x]))
= (lambda (w) (((lambda (a) y) w) z))
= (lambda (w) (y [w <- a] z))
= (lambda (w) (y z))
```

4

```
((lambda (x) (lambda (y) ((lambda (x) (z x)) (lambda (y) (z y)))))
= (lambda (y) ((lambda (x) (z x)) (lambda (y) (z y)))) [(lambda (y)
= (lambda (y) ((lambda (x) (z x)) (lambda (y) (z y))))
```

```

= (lambda (y) (z x) [(lambda (y) (z y)) <- x]) ; reduce
= (lambda (y) (z [(lambda (y) (z y)) <- x] x [(lambda (y) (z y)) <-
= (lambda (y) (z (lambda (y) (z y)))))

```

#### 4.2.2

```

(define beta-redex?
  (lambda (exp)
    (cases lcexp exp
      (application (rator rand)
        (cases lcexp rator
          (abstraction (param body) #t)
          (else #f)))
      (else #f))))

```

#### 4.2.3

```

(define substitute
  (lambda (e m x)
    (cases lcexp e
      (varref (y) (if (eq? y x) m (varref y)))
      (application (f g)
        (application (substitute f m x)
          (substitute g m x)))
      (abstraction (y e1)
        (cond ((eq? y x) (abstraction y e1))
              ((not (free? x e1)) (abstraction y e1))
              ((not (free? y m))
               (abstraction y
                 (substitute e1 m x)))
              (else (let ((z (next-symbol)))
                      (abstraction z
                        (substitute (substitute e1 m x) z m x)))))))

```

m  
x))

#### 4.2.4

```
(define beta-reduce
  (lambda (beta-redex)
    (cases lcexp beta-redex
      (application (abst m)
        (cases lcexp abst
          (abstraction (x e)
            (substitute e m x))
          (else (eopl:error "not a redex"))))
      (else (eopl:error "not a redex")))))
```